

Vibe Coding & the Trust Gap

The Psychology of Developer Trust in AI-Generated Code

Cyber Psychology Portfolio Project
By Jessica Kyle

Research Question: How does the speed and fluency of AI-generated code change developers' psychology in relation to review, ownership, and risk?

1. Executive Summary

"Vibe Coding," a term coined in early 2025 to describe building software by specifying what you want in natural language and letting an AI model write the implementation, has moved from a novelty to a mainstream way real products are built, including products that handle financial data, health information, and credentials. This report examines a specific, under-examined risk in the shift: not that AI-generated code contains bugs, which is expected of any code this way, but that the experience of generating code this way changes how the person directing it relates to review, ownership, and risk.

Independent research converges on a consistent picture: AI-generated code introduces security vulnerabilities in roughly 40-45% of security-sensitive tasks across major tools and models, and this rate has not meaningfully improved as the tools have matured. More strikingly, a controlled study by Stafford found that developers using AI coding assistance not only wrote less secure code but also rated it as more secure than code written without AI assistance. That combination of lower actual security paired with higher perceived security is as much a psychological finding as a technical one, and it is the through-line of this report.

2. Background: What Changed

Vibe Coding differs from traditional AI-assisted development (autocomplete, call suggestions, line-by-line review) in degree and intent: The person directing the AI is often, and understandably, encouraged to stop reading the generated code closely and evaluate the results by whether it works, not by how it works. Industry-wide scanning backs up how far this has scaled and what it has cost. A large-scale scan of over 5,600 publicly deployed applications built with popular AI coding tools found that roughly one in three shipped with exploitable vulnerabilities, including hundreds of exposed API keys and credentials and over 175 instances of exposed personal data, such as bank account information. AI-assisted code commits leak hard-coded secrets at roughly twice the rate of traditional written commits. A dedicated academic tracker at Georgia Tech documented a rapid month-over-month rise and confirmed vulnerabilities traceable specifically to AI-generated code through early 2026. Researchers there believed the true count is several times higher than what has been formally cataloged.

These numbers describe outcomes. The remainder of the support focuses on the psychological mechanism that reliably produces them.

3. Psychological Framework

3.1 The Fluency Heuristic

People routinely judge how correct something is by how smoothly it reads, a well-documented shortcut called processing fluency. AI-generated code, stylistically confident and consistent, will be formatted, plausibly named, free of typos and hesitation marks that signal a human still working something out. That surface fluency is processed automatically as a signal of underlying correctness, even though the two are unrelated. A vulnerable function can look, at a glance, more trustworthy than a correct one written by a tired human.

3.2 Automation Bias and the Confidence Inversion

Automation bias is the tendency to over-trust output from an automated system precisely because it is automated. The Stanford finding described above is a textbook case: participants using AI assistance produce measurably less secure code. You reported greater confidence in its security than participants who wrote code unassisted. The AI's fluency and speed were read as competence, inverting the relationship between actual and perceived risk.

3.3 Diffusion of Responsibility

When a person writes flawed code themselves, the flaw feels like their mistake to find and fix. When an AI generates the code, authorship becomes ambiguous, and psychological ownership is felt, since the sense that something is "mine to be responsible for" diffuses along with it. This does not require a conscious decision to skip review; Diffusion of responsibility operates below that level, quietly lowering the sense of urgency to check work that doesn't feel fully one's own.

3.4 Flow-State Disengagement, by Design

The person who popularized the term vibe coding described the practice as giving in to the flow of the process and largely setting aside close attention to the underlying code. Flow states are associated with reduced analytical, deliberate processing, which is useful for creative momentum but structurally incompatible with the kind of skeptical line-by-line scrutiny that catches a broken authorization check. A vibe-coding appeal and its central risk stem from the same source: it is designed to make the system 2 review step seem unnecessary.

3.5 Present Bias and Velocity Incentives

Shipping a working feature today is immediately visible and rewarded by users, by managers, by the dopamine of a deploy that works. A security review is effortful, invisible when it succeeds,

and its benefit (a breach that didn't happen) is never observed. This is the same present-biased pattern documented elsewhere in this portfolio's password and MFA findings. Now apply to engineering culture: velocity is the certain, immediate payoff, and security is the uncertain, deferred one.

3.6 The Absent Mental Model

Skepticism requires knowing what to be skeptical of. A citizen developer or an early-age founder building an app end-to-end with an AI tool often has no prior mental model of concepts like access control, credential storage, or injection risk, so no internal alarm ever fires when the AI's output happens to get one of them wrong. This differs from an experienced engineer skipping a review they know they should do; it is the absence of the knowledge that a review is warranted at all.

4. Case Study: The Tea App Breaches

In July 2025, the women staying safely at tea suffered two breaches and quick sessions. The first exposed roughly 7,200 images from a legacy, unsecured cloud storage bucket, including around 1,300 identity-verification selfies and government IDs. Days later, a second issue was discovered: over 1.1 million private direct messages exchanged between users from 2023 through mid-2025 were exposed, many of which contained names, phone numbers, and highly sensitive personal disclosures. An independent technical review afterward found API keys and access tokens hard-coded directly into the app source code and backend endpoints that allow direct, authenticated client access to user data, a basic access-control failure rather than a sophisticated exploit.

Tea's developers were widely reported to have relied heavily on AI-assisted, vibe-coded development to build and ship features quickly, without the structured code review or security architecture a security-sensitive consumer app of its scale typically requires. Whatever the exact division of AI versus Hillman authorship, the incident is now a widely cited industry example of what happens when the psychological pattern in Section 3 fluent output, diffuse ownership, and velocity over verification plays out, and real, publicly deployed product handling sensitive personal data.

5. Fictional Application: Elite Healthcare

Applying this to Elite Healthcare, the fictional organization used elsewhere in this portfolio: an operations team, under pressure to launch a new patient self-intake portal ahead of a deadline, uses an AI app-building tool to generate the portal in days rather than the months a traditional build would take. The tool produces a clean, functional interface, and the team is confident because the demo works flawlessly, leading them to push it to production without a security or compliance review, reasoning that the underlying code was AI-generated and therefore

presumably followed best practices. The generated backend, in fact, includes an access-control flaw that would allow one patient's session to retrieve another patient's intake data. The demo would not have caught this direct HIPAA exposure because the demo never tested for it.

6. Analysis: Where the Trust Gap Forms

Stage	What Happens	Psychological Mechanism
Prompting the build	The developer describes the desired outcome, not the implementation	Ownership begins to diffuse before any exists
Reviewing the output	Code looks clean, confident, and well-structured	Fluency heuristic polish is red as correctness
Testing	The demo works as intended	Automation bias: Functional success is mistaken for security success
Deciding to ship	Deadline pressures favor moving forward	Present bias, which is velocity, is the certain, felt reward
Post-incident review	The root cause is a basic, Well-known flaw class	Absent a mental model, no one was positioned to catch it

7. Recommendations

7.1 Individual-Level (Developers & Citizen Builders)

- Treat a working demo in a secure application as two separate claims, each requiring its own verification. "It runs" isn't evidence of access control, credential handling, or input validation.
- Deliberately reintroduce a review step even when it doesn't feel necessary; The absence of the urge to review itself is the signal automation buys, not evidence that review is unneeded.
- For non-security-trained builders, use a fixed checklist of known AI-coding failure classes (hard-coded secrets, missing authorization checks, authenticated endpoints). Rather than relying on intuition to flag a problem you may not have a mental model for.

7.2 Organizational-Level

- Require a security review for any AI-generated code reaching production that handles sensitive data, regardless of how confident the demo made the team feel; treat AI authorship as a risk flag that triggers review, not a substitute for it.
- Integrate automated secret scanning and static analysis directly into the AI-assisted development pipeline, since AI-assisted commits leak credentials at roughly twice the rate of traditional written code.
- Extend this portfolio's AI risk-management framework (see this candidate's GRC portfolio) to specifically cover AI-assisted software development, not only AI decision-making tools; the same governance questions (oversight, accountability, verification) apply to code and AI rights.
- Build velocity incentives that include a security gate, so shipping fast and shipping reviewed code are not experienced as competing goals by the team doing the building.
- Provide citizen developers and non-engineer team building with AI tools a lightweight, mandatory security checklist before any production deployment, rather than assuming security literacy that may not exist.

8. Conclusion

The risk in AI-generated code is not simply that models make mistakes; all code has bugs. It is that the experience of generating code this way is specifically structured to reduce the scrutiny that would normally catch them: fluent output reads as correct; authorship feels diffused; Flow State is discouraged; reading is rewarded immediately, while security is not rewarded at all until it fails. Closing the gap does not require distrusting AI coding tools wholesale; it requires building review and verification back into a workflow that was, by its own popularized description, designed to let people stop paying attention to the code underneath.